Exploring Transformer Architectures for Character-Level Language Modeling

https://github.com/jhqiu21/CharacterLLM

Liu Yuxin* A0288407U e1308816@u.nus.edu

Qiu Jinhang* A0288507R qiujinhang@u.nus.edu

Tian Junjie* A0266079R junjietian@u.nus.edu

Yu Hanyue* A0255565W e0960235@u.nus.edu

***Abstract*—This work investigates character-level language modeling on the text8 corpus using two representative sequence modeling architectures: a recurrent LSTM and a compact decoder-only Transformer. Beyond the baseline comparison, we conduct a structured exploration of how architectural design choices, positional encoding strategies, training hyperparameters, and extended optimization influence model performance under a small parameter budget. Additional analyses, including alternative tokenization schemes, diversity-oriented evaluation metrics, and modified training objectives, offer further insight into the behavior and limitations of lightweight sequence models. Our findings provide a coherent empirical picture of how different modeling decisions interact in this setting, highlighting practical considerations for building effective small-scale Transformers.**

I. Introduction

The Transformer architecture [4] has become the foundation of modern Natural Language Processing, powering models such as GPT [6] that demonstrate remarkable generative capabilities. While these systems operate at massive scales, understanding their core mechanisms remains valuable even in significantly smaller settings [1], [2]. In this project, we implement a lightweight decoder-only Transformer and compare it with an LSTM baseline [3] to study how different sequence modeling paradigms behave in a controlled, character-level prediction task.

A. Character-Level Language Modeling

Character-level modeling requires a system to predict the next symbol given a fixed-length context. The task is formally expressed as estimating

$$P(x_{t+1} \mid x_{t-L+1}, \ldots, x_t), \tag{1}$$

where each x_i is a character drawn from a small vocabulary. This formulation avoids the confounding effects of word- or subword-level tokenization and provides a clean environment for examining how models learn long-range dependencies.

B. Dataset

We use the text8 corpus, a widely adopted benchmark consisting of 100 million lowercase English characters restricted to 27 distinct tokens. The dataset offers both simplicity and sufficient sequence length to reveal differences in generalization, optimization behavior, and context utilization across modeling approaches.

C. Motivation and Approach

This study investigates the capabilities and limitations of two representative sequence modeling paradigms: a recurrent LSTM [1], [2] and a minimal decoder-only Transformer [4]. We aim to understand how self-attention in Transformers helps capture long-range dependencies compared to sequential updates in LSTMs, and how architectural and training choices affect model performance. Through systematic exploration of hyperparameter configurations, we identified combinations that yield strong performance for each model and conducted extended training experiments using these optimized settings [6].

D. Evaluation and Contribution

We evaluate model performance using cross-entropy loss and token-level accuracy on validation data [3]. Generated sequences are also inspected qualitatively to assess coherence and plausibility. This study provides a structured exploration of character-level language modeling, highlighting how architectural and training choices influence both accuracy and sequence quality. The results offer practical guidance for implementing small-scale Transformers and contribute to broader understanding of sequential modeling paradigms [4], [6].

II. Background and Model Architecture

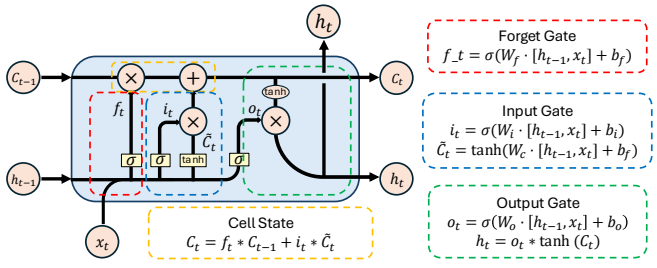


Fig. 1. LSTM structure. The input, forget, and output gates control the flow of information through the cell state c_t and hidden state h_t [7].

Recurrent neural networks (RNNs) and their gated extensions, most notably the Long Short-Term Memory (LSTM)

*Authors are listed in alphabetical order by surname; authorship order does not imply contribution ranking.

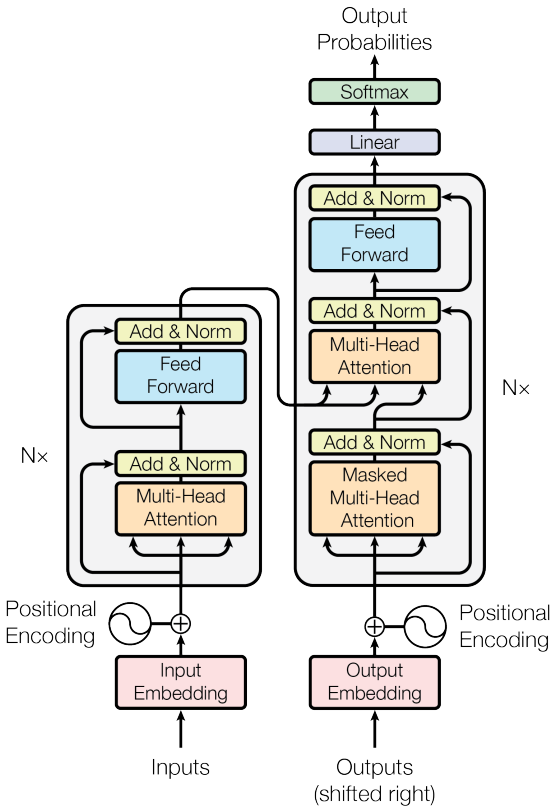


Fig. 2. Transformer model architecture, consisting of stacked multi-head self-attention and feed-forward layers with residual connections and layer normalization [4].

architecture, have long been used for sequential data modeling. LSTMs mitigate the vanishing and exploding gradient problems of simple RNNs by introducing gated memory cells that can preserve information over long time spans [1], [2]. They achieved strong performance in early sequence-to-sequence applications, such as neural machine translation, where stacked encoder–decoder LSTMs were used to map entire input sentences to output sequences [3]. However, the inherently sequential nature of recurrent computation limits parallelization and makes it difficult to efficiently capture very long-range dependencies [4].

To overcome these limitations, Vaswani et al. (2017) introduced the Transformer architecture, which replaces recurrence with self-attention to compute interactions between all tokens in a sequence in parallel [4]. Self-attention allows direct modeling of global dependencies, while enabling fully parallel computation during training. A Transformer layer consists of a multi-head attention module followed by a position-wise feed-forward network (FFN), with residual connections and layer normalization around each sublayer to stabilize optimization.

A. Self-Attention Mechanism

The self-attention mechanism projects each input token into three vectors: a query Q, a key K, and a value V. Attention scores are obtained by the scaled dot-product of queries and keys, normalized through a softmax function, and used to compute weighted sums of the value vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2}$$

This operation allows each token to incorporate contextual information from all other tokens in the sequence, in contrast to the step-by-step updates of recurrent networks.

B. Multi-Head Attention

Instead of computing a single attention distribution, the Transformer employs multiple heads to capture different types of contextual relations in parallel. Each head uses distinct linear projections W_i^Q, W_i^K, W_i^V to produce its own attention output. The outputs are concatenated and linearly transformed through W_i^O

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{3}$$

Multi-head attention enables the model to attend to information at different positions and representation subspaces simultaneously.

C. Feed-Forward Network and Normalization

Following the attention block, each position independently passes through a fully connected feed-forward network (FFN) consisting of two linear layers with a non-linear activation, typically ReLU or GELU:

$$\text{FFN}(x) = W_2\sigma(W_1x + b_1) + b_2 \tag{4}$$

where the hidden layer usually expands the model dimension d_{model} by a fixed factor. Layer normalization and residual connections are applied around both the attention and feed-forward sublayers to stabilize training and facilitate gradient flow in deep stacks [5].

D. Positional Encoding

Since the self-attention operation is permutation-invariant, positional information must be injected explicitly. The original Transformer introduces *absolute positional encodings*, either as fixed sinusoidal functions or as learned embedding tables added to the token embeddings [4]. Fixed sinusoidal encodings assign a unique vector to each position while enabling the model to infer relative distances through linear combinations, whereas learned positional embeddings treat positions as trainable parameters.

More recent work has explored *relative* and *rotary* positional schemes, which encode position differences directly into the attention mechanism and often generalize better to longer contexts. Examples include relative position biases, rotary position embeddings (RoPE), and attention with linear biases such as AliBi.

We also try to adapt a pure relative positional encoding scheme, implemented as learnable per-head relative position biases added directly to the attention logits rather than to the input embeddings. We empirically compare several representatives from both families; the experimental study and our final choice of positional encoding are presented in Section IV.

E. Decoder-Only Architecture for Language Modeling

The original Transformer includes both an encoder and a decoder for sequence-to-sequence tasks such as translation. For autoregressive language modeling, only the decoder component is required. The decoder-only formulation retains masked self-attention and feed-forward sublayers, omits the encoder and encoder–decoder attention, and uses a causal mask so that each position attends only to preceding tokens. This design, adopted by GPT-style models [6], aligns naturally with next-character prediction and is the basis of the Transformer architecture studied in this work.

III. Experimental Configuration

The experiments aim to assess how architectural and optimization choices influence the performance of the decoder-only Transformer on the character-level language modeling task, and to compare its behavior with that of a recurrent Long Short-Term Memory (LSTM) baseline trained under identical conditions.

A. Experimental Setup

*1) **Dataset Split and Preprocessing**:* We use the TEXT8 corpus for all experiments. The 100M-character stream is split into 85.5M characters for training, 4.5M for validation, and 5M for testing. Following the standard setup, each of the 27 distinct characters (the letters 'a'–'z' and space) is mapped to a unique integer token ID, yielding a fixed vocabulary without any additional normalization or filtering.

For sequence modeling, the training stream is segmented into fixed-length sequences of length $T = 128$. The same preprocessing pipeline (tokenization, splitting, and batching) is used for both the Transformer and LSTM models to ensure a fair comparison.

*2) **Implementation Details**:* Our implementation follows a decoder-only Transformer as described in Section II-E. The baseline configuration, which serves as the starting point for subsequent architectural and optimization ablations, is defined as follows:

*a) **Model Size**:* We use a hidden dimension of $d_{\text{model}} = 256$, with $N_{\text{layers}} = 4$ decoder blocks, $N_{\text{heads}} = 4$ attention heads, and a per-head dimensionality of $d_{\text{head}} = 64$.

*b) **Optimizer**:* Optimization is performed with AdamW with a fixed hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\varepsilon = 10^{-8}$. A weight decay coefficient of 0.1 is applied to all weights except for biases and LayerNorm parameters.

*c) **Learning Rate Schedule**:* The learning rate follows a linear warmup followed by cosine decay. Unless otherwise specified, the initial learning rate is set to 3×10^{-4} with a warmup duration of $2{,}000$ optimization steps.

*d) **Training Parameters**:* The global batch size is $B = 128$, and the sequence length is fixed to $T = 128$.

*3) **Evaluation Metrics**:* We report a set of likelihood-based, accuracy-based, and generation-quality metrics on both the validation and test sets. Unless otherwise stated, model selection is performed based on the validation average cross-entropy loss (denoted *loss_all*). All other metrics are reported for analysis on the held-out test set.

*a) **Likelihood-based metrics**:* Our primary objective is the average cross-entropy loss $\mathcal{L}_{\text{avg}}$ over all position,

$$\mathcal{L}_{\text{avg}} = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} \left(-\log p_{b,t,y_{b,t}} \right) \tag{5}$$

We also report bits-per-character (BPC) and perplexity (PPL), derived as

$$\text{BPC} = \mathcal{L}_{\text{avg}} / \ln 2 \tag{6}$$

$$\text{PPL} = \exp(\mathcal{L}_{\text{avg}}) \tag{7}$$

*b) **Accuracy-based metrics**:* We report average token accuracy Acc_{avg} over all positions and last-position accuracy Acc_{last} focusing on the final token of each sequence, which is often the most challenging to predict and serves as an indicator of the model's ability to capture long-range dependencies.

To probe performance on different frequency regimes, we further decompose accuracy into *common-token accuracy* $\text{Acc}_{\text{common}}$ and *rare-token accuracy* Acc_{rare}. Tokens are ranked by their empirical frequency in the training corpus, and we split them into a head (common) and tail (rare) partition; accuracy is then computed separately on positions whose ground-truth token falls into each partition.

*c) **Calibration**:* To assess probabilistic calibration, we compute the expected calibration error (ECE) by binning predictions by confidence and averaging the gap between empirical accuracy and predicted confidence.

*d) **Diversity and semantic quality**:* For generated text, we evaluate diversity and structural quality using three complementary metrics.

- ***Self-BLEU:*** Self-BLEU measures the internal diversity by treating one generated sample as the candidate and the remaining samples as references, and computing the n-gram overlap. Higher Self-BLEU indicates lower diversity and a stronger tendency toward repetitive patterns.
- ***Distinct-N:*** Distinct-N quantifies lexical diversity as the ratio of unique N-grams to the total number of N-grams. Higher Distinct-N indicates fewer repetitions.
- ***Coherence Score:*** We define a scalar coherence score as the average of a repetition penalty and a word-length regularity term. This metric penalizes short substring reuse while rewarding samples with a mean word length near a target (e.g., five characters), serving as a proxy for natural structural statistics without excessive repetition. Higher coherence score indicates fewer repetitions and better structural coherence.

B. Hardware and Software Environment

All experiments were conducted on a single Linux server equipped with eight NVIDIA A5000 GPUs and a multi-core CPU. Models were implemented in Python using JAX and Flax with CUDA support. When multiple GPUs were used, training employed data parallelism across devices. A common infrastructure for checkpointing and logging was shared across all runs, and the same hardware–software stack was used for all Transformer and LSTM experiments.

C. Modified Training Objective

For the extended training, we use a modified training loss $\mathcal{L}_{\text{train}}$ that combines label smoothing, position weighting, and an optional entropy bonus:

$$\mathcal{L}_{\text{train}} = \underbrace{\frac{1}{B}\sum_{b=1}^{B}\sum_{t=1}^{T}\tilde{w}_t\,\mathrm{CE}(\mathbf{q}_{b,t},\mathbf{p}_{b,t})}_{\mathcal{L}_{\text{weight}}} - \lambda\overline{\mathcal{H}} \tag{8}$$

where $\mathrm{CE}(\mathbf{q},\mathbf{p})$ is the token-level cross-entropy between target distribution $\mathbf{q}$ and model prediction $\mathbf{p}$.

*1) **Label smoothing**:* The smoothed target distribution $\mathbf{q}_{b,t}$ is defined as

$$\mathbf{q}_{b,t} = (1-\varepsilon)\,\mathbf{e}_{y_{b,t}} + \frac{\varepsilon}{V}\,\mathbf{1}, \tag{9}$$

with label-smoothing coefficient $\varepsilon = 0.05$, $\mathbf{e}_{y_{b,t}}$ the one-hot vector for the ground-truth character $y_{b,t}$, and V the vocabulary size.

*2) **Position weighting**:* To place slightly more emphasis on the tail of each sequence while preserving supervision at all positions, we apply normalized position weights $\tilde{w}_t$:

$$\tilde{w}_t = \frac{w_t}{\sum_{s=1}^{T} w_s} \qquad \sum_{t=1}^{T}\tilde{w}_t = 1 \tag{10}$$

*3) **Entropy bonus**:* Finally, we encourage less peaked output distributions via an entropy bonus. The per-token entropy is

$$\mathcal{H}(\mathbf{p}_{b,t}) = -\sum_{v=1}^{V} p_{b,t,v}\log p_{b,t,v}, \tag{11}$$

and its minibatch average is

$$\overline{\mathcal{H}} = \frac{1}{BT}\sum_{b=1}^{B}\sum_{t=1}^{T}\mathcal{H}(\mathbf{p}_{b,t}). \tag{12}$$

This modified objective $\mathcal{L}_{\text{train}}$ is used *only* for optimization in extended experiments. All validation and test metrics (loss, BPC, perplexity, accuracies, and ECE) are computed using the standard unweighted cross-entropy $\mathcal{L}_{\text{avg}}$.

D. Optimization and Training Dynamics

Transformer training is highly sensitive initially, largely due to Layer Normalization and residual connections. The warm-up period is crucial for stability, preventing convergence failure by gradually increasing the learning rate from zero to stabilize parameters before large gradient steps, as noted by Liu et al. (2020) [5]. We performed a small grid search over learning rates in $\{2\times10^{-4}, 3\times10^{-4}, 10^{-3}\}$ and warm-up lengths in $\{0, 2{,}000, 4{,}000\}$ steps for the baseline Transformer. The most robust configuration was an initial learning rate of 3×10^{-4} with a 2,000-step warm-up, which is thus adopted as our default setting for extended training experiments.

E. Architectural Configurations

We systematically varied structural parameters to investigate the trade-off between model capacity and computational cost under a fixed training budget. Unless otherwise stated, the default configuration adopted in our main experiments is specified in this subsection, while the empirical justification for these choices is provided in the preliminary ablations of Section IV.

*1) **Model Capacity**:* Model capacity is primarily controlled by the number of Transformer blocks N_{layers}, the hidden size d_{model}, and the allocation of attention heads.

*a) **Depth and MLP expansion**:* Through our experiments with Transformer depths $N_{\text{layers}} \in \{2,4,6\}$ and MLP expansion ratios $r_{\text{MLP}} \in \{4,6\}$, we found that a 4-layer Transformer with an MLP expansion ratio of 4 consistently achieves the best trade-off between loss and accuracy under our training budget. Deeper stacks tend to degrade performance, while larger MLPs provide only marginal gains. Therefore, we adopt a 4-layer backbone with $r_{\text{MLP}} = 4$ as our default architecture for all extended training experiments.

*b) **Width and attention allocation**:* We vary the hidden size between $d_{\text{model}} = 256$ and $d_{\text{model}} = 384$, respecting the constraint $d_{\text{model}} = N_{\text{heads}} \cdot d_{\text{head}}$. At $d_{\text{model}} = 256$ we consider attention factorizations $(N_{\text{heads}}, d_{\text{head}}) \in \{(8,32),(4,64)\}$, and at $d_{\text{model}} = 384$ we additionally explore $(6,64)$. In the main extended training experiments we use $d_{\text{model}} = 256$ with $(N_{\text{heads}}, d_{\text{head}}) = (4,64)$ as the default width and attention configuration; the empirical comparison of these choices is reported in Section IV.

*2) **Batch Size (B) and Sequence Length (T)**:* Batch size B and context length T jointly determine the memory footprint and the number of tokens processed per parameter update. We consider $B \in \{128\}$ and $T \in \{64, 128, 256\}$, with $(B,T) = (128,128)$ as our default configuration. Increasing B improves the stability of the gradient estimate but linearly increases memory usage, while increasing T provides more context to each prediction at the cost of the quadratic $\mathcal{O}(T^2)$ scaling of self-attention. As shown in Table I, a context length of $T = 128$ achieves the best validation BPC and last-position accuracy. Shorter contexts ($T = 64$) underperform due to insufficient history, whereas longer contexts ($T = 256$) slightly hurt performance under our fixed compute budget, likely because each parameter update sees fewer distinct subsequences. We therefore set $T = 128$ as the default context length in the extended training experiments.

TABLE I
ABLATION ON CONTEXT LENGTH T. ALL MODELS SHARE THE SAME CONFIGURATION EXCEPT FOR T. WE REPORT VALIDATION CROSS-ENTROPY (BPC), AVERAGE TOKEN ACCURACY $\mathrm{Acc}_{\mathrm{AVG}}$ AND LAST-POSITION ACCURACY $\mathrm{Acc}_{\mathrm{LAST}}$.

T	BPC ↓	$\mathrm{Acc}_{\mathrm{avg}}$ (%) ↑	$\mathrm{Acc}_{\mathrm{last}}$ (%) ↑
64	2.161	53.05	58.01
128	**2.128**	**54.60**	**58.89**
256	2.215	54.00	57.13

TABLE II

PRELIMINARY EXPERIMENTS ON *text8*. SHARED SETTINGS: SEQUENCE LENGTH T=128, BATCH B=128, N_{LAYERS}=4, MLP RATIO = 4, ADAMW OPTIMIZER WITH WEIGHT DECAY 0.1. WE REPORT VALIDATION BEST LOSS, BEST LAST-CHARACTER ACCURACY, AND BEST OVERALL ACCURACY

					Dropout Rate				**Performance Metrics**			
ID	d_{model}	N_{heads}	lr	Warmup	Emb	Attn	Resid	MLP	Best Loss	Best $\text{Acc}^{\text{a}}_{\text{last}}$	Avg. Acc_{last}±SD	Best $\text{Acc}_{\text{total}}$
E-beta	256	4	0.0003	0	0.0	0.0	0.0	0.0	1.3425	62.21%	59.76%±1.41%	57.90%
E0	256	4	0.0003	2000	0.1	0.1	0.1	0.1	1.4523	59.08%	57.11%±1.00%	54.84%
E1	256	4	0.0002	2000	0.1	0.1	0.1	0.1	1.5005	58.01%	56.41%±1.74%	53.96%
E2	256	4	0.0004	2000	0.1	0.1	0.1	0.1	1.4274	60.94%	57.81%±1.77%	55.67%
E6	384	6	0.0003	2000	0.1	0.1	0.15	0.2	1.3649	63.67%	59.53%±1.58%	57.60%
E8	256	4	0.0003	2000	0.0	0.0	0.0	0.0	1.3684	61.91%	60.19%±1.97%	57.53%

*3) **Dropout Regularization**:* We ablate dropout rates $\{0.0, 0.1, 0.15, 0.25\}$ applied at four locations in the Transformer stack (token embeddings, attention outputs, residual connections, and MLP blocks). Motivated by the preliminary results in Section IV, and consistent with the intuition that this small, relatively clean corpus does not strongly benefit from aggressive regularization, our extended training experiments focus on two regimes: a no-dropout configuration (dropout $= 0.0$) and a moderately regularized configuration (dropout $= 0.1$). Higher dropout rates are only used in short ablation runs and are not included in the main comparisons.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Preliminary experiments

We first run a series of short preliminary experiments on TEXT8 to coarsely tune optimization, regularization, and width. We highlight the configurations that shaped our later large-scale experiments.

*1) **New baseline**:* The initial model struggled to form coherent words or meaningful sentences. To address this, E-beta increased sequence length T and adjusted N_{layers} and N_{heads}. It achieves strong validation likelihood (best loss 1.34) and last-step accuracy (62.21%) and could generate coherent words and complete sentences, though still having spelling mistakes and repetitive patterns. E-beta is hence served as a stronger reference configuration for subsequent tuning, but we continue refining optimization and regularization before fixing our final baseline.

*2) **Learning rate and warm-up**:* Starting from the E-beta configuration, we introduce a $2{,}000$-step linear warm-up and sweep the learning rate $lr \in \{2{\times}10^{-4}, 3{\times}10^{-4}, 4{\times}10^{-4}\}$ (E1, E0, E2 in Table II). A rate of $3{\times}10^{-4}$ with warm-up (E0) provides a good compromise between stability and speed: lowering the rate to $2{\times}10^{-4}$ (E1) slows optimization and yields the worst best loss (1.50) and last-position accuracy (58.01%), while increasing it to $4{\times}10^{-4}$ (E2) improves best loss (1.43) and last-position accuracy (60.94%) but exhibits larger variance across checkpoints. These findings support our setting of $lr = 3{\times}10^{-4}$ with $2{,}000$-step warm-up in subsequent extended training experiments.

*3) **Width and attention allocation**:* To assess the benefit of wider models, E6 increases the hidden size to $d_{\text{model}} = 384$ with $(N_{\text{heads}}, d_{\text{head}}) = (6, 64)$ and slightly stronger dropout. This wider model attains the highest best last-position accuracy among the preliminary runs (63.67%) and a competitive best loss (1.36). However, the gains over the $d_{\text{model}} = 256$ baselines (E0, E8) are modest compared to the additional parameters and per-step compute. Under our fixed training budget we therefore adopt $d_{\text{model}} = 256$ with $(N_{\text{heads}}, d_{\text{head}}) = (4, 64)$ as the default width and head configuration for all extended experiments.

*4) **Dropout regularization**:* For the same 4-layer, $d_{\text{model}} = 256$ architecture, we ablate dropout strength applied at all four locations in the Transformer stack (token embeddings, attention outputs, residual connections, and MLP blocks). Comparing E0 (dropout 0.1) and E8 (no dropout) shows that dropout is detrimental in this small, clean character-level setting: E8 attains a better best loss (1.37 vs. 1.45), higher best last-position accuracy (61.91% vs. 59.08%), and the highest average last-position accuracy ($60.19\% \pm 1.97$). Additional runs with stronger dropout (e.g., rate 0.15–0.25, not shown in Table II) further degrade validation BPC and accuracy, suggesting that standard dropout acts primarily as harmful noise rather than useful regularization on TEXT8.

Compared to E-beta, E8 trades a slightly worse best loss for improved stability and the best average last-position accuracy. We therefore use E8 as the reference architecture when studying positional encodings (Section IV.B). For the extended long-run experiments, we further adopt E2 with RoPE positional encoding and E8 with relative positional encoding as the bases for TR2o and TR8o, respectively.

B. Proposed Model Enhancements

*1) **Positional Embedding**:* We first investigate which positional encoding is most suitable for our character-level decoder. Table III compares five alternatives on top of the E8 configuration: the original learned encoding, fixed sinusoidal encoding, a hybrid variant that adds a small learned offset, rotary positional embeddings (RoPE) [10], and AliBi [11]. All models are identical except for the choice of positional encoding, and we report validation best BPC, best perplexity, and best last-position accuracy Acc_{last}.

RoPE clearly provides the best likelihood-based metrics, reducing BPC from 2.0596 (learned) to 1.9559 and perplexity from 4.1687 to 3.8795. This corresponds to a gain of +2.29 percentage points in Acc_{last} (from 60.11% to 62.40%),

while using fewer parameters than the learned and hybrid variants. AliBi achieves a similar Acc_{last} but slightly worse BPC and perplexity, indicating that its predictions are less well calibrated as a language model. Given these consistent improvements and the fact that RoPE does not increase the model size, we adopt RoPE as the default positional encoding in all subsequent experiments.

TABLE III
ABLATION ON DIFFERENT POSITIONAL ENCODING METHODS.

Type	BPC ↓	Perplexity ↓	Acc_{last} (%) ↑	#Params
Learned	2.0596	4.1687	60.11	3,202,048
Sinusoidal	2.0898	4.2570	58.98	3,169,280
Hybrid	2.0607	4.1718	60.16	3,202,048
RoPE	**1.9559**	**3.8795**	**62.40**	3,169,280
AliBi	1.9984	3.9956	62.40	3,169,280

*2) **Relative Positional Embedding**:* Motivated by the benefits of incorporating relative position information, we adopt a pure relative positional embedding (RelPos) scheme in our character-level decoder. To evaluate its effect, we integrate RelPos into the previously high-performing E2, E6, and E8 configurations and perform short-run experiments in 10k steps. Table IV summarizes the validation metrics, including BPC, perplexity, last-character accuracy, and number of parameters, measured at the checkpoint with minimum validation BPC.

TABLE IV
ABLATION ON PURE RELATIVE POSITIONAL EMBEDDING APPLIED TO SELECTED CONFIGURATIONS.

ID	BPC ↓	Perplexity ↓	Acc_{last} (%) ↑	#Params
E2	2.1115	4.3213	57.71	3238640
E6	2.1440	4.4197	58.50	7217256
E8	**2.0686**	**4.1948**	**61.43**	3238640

Among the three configurations, E8 with relative positional embedding achieves the best validation metrics, reducing BPC to 2.0686 and perplexity to 4.1948, while increasing last-character accuracy to 61.43%. Both E2 and E6 show modest improvements in some metrics, but remain lower than E8. These results suggest that integrating pure relative positional information is most effective when combined with an already strong baseline configuration. Consequently, we adopt E8 + RelPos as the default positional encoding for subsequent long-run experiments.

*3) **N-Gram**:* In preliminary experiments, we observed a high degree of repetitions in the generated text, often taking form in short phrases or character sequences that reappear multiple times in sequence. To address this issue, we incorporated an n-gram blocking mechanism into the decoding procedure. It constrains the model by preventing it from generating any n-gram that has already appeared earlier in the same output sequence. When baseline generations contain frequent short-range repetitions, we observed that n-gram blocking effectively prevents the strong repetitive loops present in the unconstrained model. However, when the original output is only mildly repetitive, imposing a small n (e.g., 2) becomes overly restrictive and forces the model into low-probability transitions, producing long, non-English character sequences despite eliminating repetition. In contrast, higher-order constraints such as 3-gram or 4-gram blocking activate far less frequently and often yield outputs nearly identical to the no-blocking case.

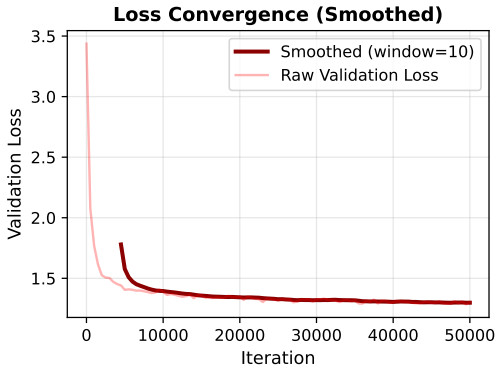


Fig. 3. Validation loss convergence for the TR2o model over 50k training iterations. The light curve shows the raw validation loss, while the dark curve is a moving average with a window size of 10. Loss decreases rapidly in the first few thousand iterations and then gradually plateaus, indicating stable convergence by the end of training.

C. Extended Training Experiments

Building upon the findings from the model enhancement section, we first determined which configurations should be selected for long-run training. The earlier analysis showed that, among all RoPE-based models, the E2 variant consistently achieved the best validation performance, while among models using relative positional encoding, E8 was the strongest. Therefore, these two configurations served as the bases for our extended experiments, denoted TR2o (RoPE) and TR8o (Relative).

Figure 4 provides the training and validation curves for the TR2 model, from which we observe that both the loss and accuracy begin to plateau after roughly 20k iterations, with only minor oscillations thereafter. This suggests that additional training beyond this point still offers room for gradual improvement, but excessively long runs would yield diminishing returns relative to the computational cost. Balancing these factors, we set the training budget for the long-run experiments (TR2o and TR8o) to 50k iterations, which provides sufficient room for refinement while keeping the total compute manageable.

Figure 3 further confirms that the 50k iterations is adequate: TR2o exhibits a rapid initial improvement followed by a smooth plateau, with no signs of divergence or late-stage instability. This validates the suitability of the 50k iteration setting for both extended runs.

Table V reports the final test-set performance of TR2o and TR8o at the checkpoints achieving minimum validation BPC. Both models share nearly identical parameter counts and training budgets, differing only in the choice of positional

TABLE V

TEST-SET PERFORMANCE OF THE EXTENDED TRANSFORMER AND LSTM MODELS TRAINED WITH THE MODIFIED OBJECTIVE. TR2O AND TR8O SHARE THE SAME ARCHITECTURE WITH E2 AND E8 AND TRAINING BUDGET BUT DIFFER ONLY IN POSITIONAL ENCODING (ROPE VS. RELATIVE POSITIONAL ENCODING). LR-1 AND LR-2 ARE LSTM BASELINES WITHOUT EXPLICIT POSITIONAL ENCODINGS, WITH LR-2 ROUGHLY MATCHING TR2O IN PARAMETER COUNT. WE REPORT BITS-PER-CHARACTER (BPC), TOKEN-LEVEL ACCURACIES (OVERALL Acc_{ALL}, LAST-POSITION Acc_{LAST}, AND ACCURACIES ON RARE AND COMMON TOKENS), EXPECTED CALIBRATION ERROR (ECE), AND SELF-BLEU-4. ALL METRICS ARE COMPUTED ON THE TEST SET AT THE CHECKPOINT WITH MINIMUM VALIDATION BPC.

ID	Pos.Enc.	Parameter	BPC ↓	Acc_{all} (%) ↑	Acc_{last} (%) ↑	Acc_{rare} (%) ↑	Acc_{comm} (%) ↑	ECE ↓	BLEU-4 ↓
TR2o	RoPE	3,169,280	1.8468	60.28	64.08	39.27	72.10	0.0830	0.0159
TR8o	Relative	3,238,640	1.8993	59.50	62.94	37.06	71.77	0.1322	0.0983
LR-1	N/A	2,115,072	1.9198	59.02	62.53	37.60	71.29	0.0227	0.1541
LR-2	N/A	3,177,052	1.8896	59.67	63.45	38.22	71.82	0.0260	0.1212

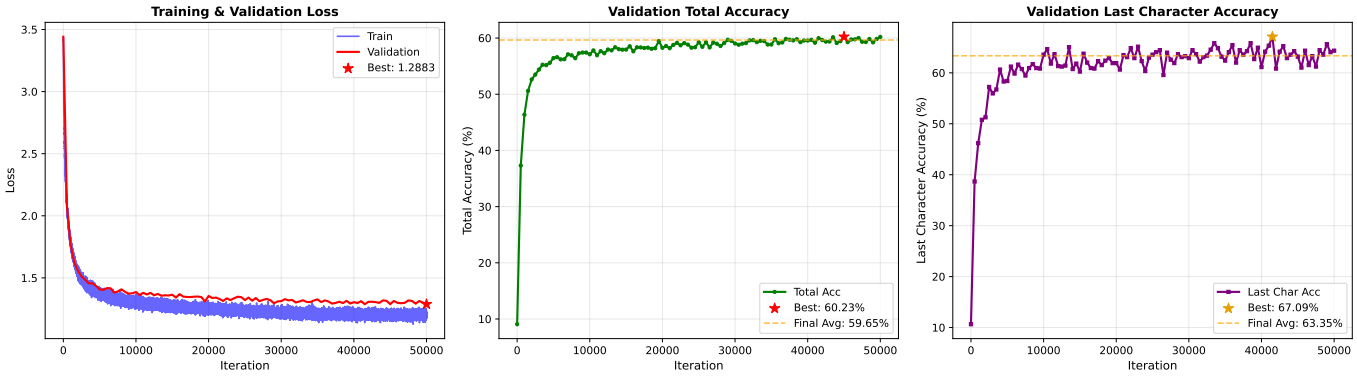


Fig. 4. Training and validation curves of the TR2 models, showing training/validation losses and corresponding validation accuracies. Stars mark the best-performing checkpoints; orange dashed lines denote final-iteration averages.

encoding. TR2o (RoPE) achieves better overall performance across all metrics: lower BPC, higher token-level accuracy, and stronger performance on both rare and common tokens. On calibration and generation diversity, TR2o also attains a substantially lower expected calibration error (0.0830 vs. 0.1322) and Self-BLEU-4 (0.0159 vs. 0.0983), indicating better calibrated probabilities and less repetitive samples. TR8o remains competitive but consistently trails TR2o across the board, and its higher ECE and Self-BLEU-4 are consistent with our qualitative observation that the relative-positional model tends to produce over-confident predictions and locally repetitive character sequences.

Overall, the extended experiments show that (1) the earlier positional-encoding trends remain valid under long-run training, with RoPE maintaining an advantage; and (2) a 50k training horizon is sufficient for stable convergence. TR2o emerges as the strongest configuration, while TR8o confirms that relative positional encoding remains viable but slightly less effective under our training setup.

D. Comparison with LSTM Baseline

To contextualize the extended Transformer results, we train two character-level LSTM baselines on TEXT8 under the same data split, batch size (B=128), sequence length (T=128), optimizer (AdamW), learning-rate schedule (warmup 2,000, $lr = 4 \times 10^{-4}$), and modified objective (label smoothing with $\varepsilon = 0.05$ and linear tail weighting) as our Transformer configuration TR2o. The only difference is the sequence modeling architecture. The smaller LSTM (LR-1) uses a 4-layer stack with hidden size 256 (2.12M parameters), while the parameter-matched variant (LR-2) increases the hidden size to 314 (3.18M parameters), comparable to TR2o's 3.17M parameters. Both LSTMs use standard token embeddings without explicit positional encodings; temporal order is modeled through the recurrent dynamics.

Table V (last two rows) reports the test-set performance of LR-1 and LR-2 alongside TR2o and TR8o. Among the Transformer models, TR2o (RoPE) is clearly stronger than TR8o (relative positional encoding), achieving lower BPC (1.8468 vs. 1.8993) and higher overall, last-position, and rare-token accuracies. Comparing TR2o with the parameter-matched LSTM LR-2, we observe that the Transformer offers consistently better likelihood and overall accuracy: TR2o improves BPC by about 0.04 (from 1.8896 to 1.8468) and increases Acc_{all} and Acc_{rare} by roughly 0.5–1 percentage point. In contrast, LR-2 attains slightly higher last-position accuracy than TR2o, and LR-1 trails both in most metrics. Overall, for a comparable parameter budget, the decoder-only Transformer provides the best trade-off between likelihood and token-level accuracy, while a well-tuned LSTM remains competitive and particularly strong on the final token.

Interestingly, calibration and diversity exhibit a different pattern. The LSTM baselines achieve substantially lower ECE values (around 0.02–0.03) than the Transformer models (0.08–0.13), suggesting better-calibrated predictive probabilities despite their worse overall likelihood. However, their

Self-BLEU-4 scores are much higher than TR2o's, indicating more repetitive generations and reduced sample diversity. TR2o simultaneously achieves strong likelihood, competitive calibration, and the lowest Self-BLEU-4, yielding the most diverse outputs among the compared models.

We also compare wall-clock efficiency under the common setting $B{=}128$, $T{=}128$ on the same $8\times$A5000 machine. TR2o and TR8o require approximately 0.027 and 0.023 seconds per optimization step, respectively, while LR-1 and LR-2 require 0.044 and 0.069 seconds per step. This corresponds to roughly 6.1×10^5 and 7.1×10^5 tokens per second for the Transformer models versus 3.7×10^5 and 2.4×10^5 tokens per second for LR-1 and LR-2. In other words, the Transformer is about $1.6\times$–$2.6\times$ faster per update despite similar parameter counts. Combining accuracy, calibration, diversity, and throughput, our extended Transformer configurations offer a strictly better overall trade-off than the LSTM baselines, although the latter remain strong and well-calibrated competitors in this small–to–medium scale regime.

V. Discussions and Future Work

A. Stability Issues in Early-Step Generation

In the TR8o model (E8 with relative positional encoding), we observed that the beginning of generated sequences sometimes exhibits instability. Typical patterns include short repeated characters such as "lll" or repeated number phrases like "one three three eight seven." In contrast, the later parts of the generation tend to show clearer syntactic structure and more coherent word formation, indicating that the model captures global patterns well but remains less stable at the initial decoding steps.

One possible remedy is to apply targeted regularization to the early timesteps of generation—for example, mild entropy smoothing or temperature scaling only at the start, or penalties that reduce short-loop token repetitions. Similar strategies have been explored in prior work on stabilizing generative decoding and mitigating early-step degeneration [12], [13]. These techniques may help prevent early-step collapse while preserving the strong long-range modeling behavior observed later in the sequence.

B. Subword Tokenization Attempt

We explored a subword tokenization setting by introducing a Byte-Pair Encoding (BPE) tokenizer trained on the corpus with a $10{,}000$-token vocabulary [14]. This setup is intended to capture frequent substrings (e.g.'able') as single units while decomposing rare or unseen words into smaller subword components.

In practice, models trained with this tokenizer showed a substantial drop in accuracy compared to the character-level setup. A likely reason would be, under a fixed vocabulary size, the model must learn representations for a wide range of token compositions. This increases the burden on a small model, which may struggle to encode highly specific or irregular word fragments effectively.

These observations suggest that BPE tokenization may not align well with the constraints of our compact character-focused architecture, and the character-level formulation remains more suitable in this setting.

C. Hybrid LSTM–Transformer Architecture

To combine the sequential inductive bias of LSTMs with the parallel efficiency and global dependency modeling of Transformers, we planed to implement a hybrid architecture in which an LSTM encoder is followed by several Transformer layers.

This improvement likely stems from the LSTM's ability to encode local structure before the attention layers refine longer-range dependencies. Similar hybrid approaches have been reported to stabilize training and improve convergence in sequence modeling tasks [8]. Although the model introduces some sequential computation, the total training time remains comparable due to the parallel nature of the Transformer layers.

Future work could explore repetition penalties or diversity-aware decoding strategies such as nucleus sampling or contrastive decoding [9] to improve semantic quality and reduce degeneracy during generation.

References

[1] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.

[2] Staudemeyer, R. C. (2019). Understanding LSTM–a tutorial into long short-term memory recurrent neural networks. arXiv preprint arXiv:1909.09586.

[3] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. Advances in neural information processing systems, 27.

[4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30

[5] Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., ... & Liu, T. (2020, November). On layer normalization in the transformer architecture. In International conference on machine learning (pp. 10524-10533). PMLR.

[6] Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training.

[7] C. Olah, "Understanding LSTMs," Blog post, Colah's Blog. Available: https://colah.github.io/posts/2015-08-Understanding-LSTMs/. [Accessed: Nov. 2, 2025].

[8] Z. Zheng, S. Huang, R. Weng, X. -Y. Dai and J. Chen, "Improving Self-Attention Networks With Sequential Relations," in IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 28, pp. 1707-1716, 2020, doi: 10.1109/TASLP.2020.2996807.

[9] Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2019). The curious case of neural text degeneration. arXiv preprint arXiv:1904.09751.

[10] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568, 127063.

[11] Press, O., Smith, N. A., & Lewis, M. (2021). Train short, test long: Attention with linear biases enables input length extrapolation. arXiv preprint arXiv:2108.12409.

[12] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In Proceedings of the 34th International Conference on Machine Learning (pp. 1321–1330).

[13] Welleck, S., Kulikov, I., Kim, J., Sun, J., Choi, Y., & Cho, K. (2020). Neural text generation with unlikelihood training. International Conference on Learning Representations (ICLR).

[14] Sennrich, R., Haddow, B., & Birch, A. (2016, August). Neural machine translation of rare words with subword units. In Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers) (pp. 1715-1725).